

138. Copy List with Random Pointer

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 17, 2026

Problem Summary

Given the head of a linked list where each node has a next pointer and a random pointer (pointing to any node or null), return a **deep copy** of the list — entirely new nodes with identical structure.

Algo

Traverse original list, for each node: `map.put(curr, new Node(curr.val))`.

Traverse again, for each node:

- `map.get(curr).next = map.get(curr.next)`
- `map.get(curr).random = map.get(curr.random)`
- Advance `curr`.

Return `map.get(head)`.

Time Complexity & Space Complexity

$O(n)$ — two passes through the list.

$O(n)$ — HashMap stores one entry per node.

Approach

Two-pass HashMap approach. Pass 1 creates a copy node for every original node and stores the mapping original → copy. Pass 2 uses the map to wire up next and random pointers for every copy node — by this point all copy nodes are guaranteed to exist in the map.

Observations

- No GROUP BY needed — we want a single row output
- Subquery calculates max earnings independently → used in WHERE to filter
- MAX() and COUNT(*) together on filtered rows gives both answers in one query

Key Insights

- Two passes are needed because random can point to any node — including ones not yet created in a single pass.
- By the end of Pass 1, **every** original node has a copy in the map — so Pass 2 can safely wire any next or random without order concerns.
- `map.get(null)` returns null in Java — so last node's next = null and any random = null are handled automatically, no null checks needed.
- `map.get(head)` returns the copy of head — the entry point to the new list.

Observations

- No null checks needed anywhere — HashMap's null key behavior handles terminal pointers cleanly.
- Original list is never modified — purely read in both passes.
- Space can be reduced to $O(1)$ using the interleaving trick (no HashMap) — weave copy nodes between original nodes, set randoms, then separate — but that's significantly more complex.

Linked List

DescriptionEditorialSolutionsSubmissions

138. Copy List with Random Pointer

Solved

MediumTopicsCompaniesHint

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or `null`.

Construct a **deep copy** of the list. The deep copy should consist of exactly n brand new nodes, where each new node has its value set to the value of its corresponding original node. Both the `next` and `random` pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. **None of the pointers in the new list should point to nodes in the original list.**

For example, if there are two nodes X and Y in the original list, where $X.random \rightarrow Y$, then for the corresponding two nodes x and y in the copied list, $x.random \rightarrow y$.

Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of `[val, random_index]` where:

- `val`: an integer representing `Node.val`
- `random_index`: the index of the node (range from `0` to `n-1`) that the `random` pointer points to, or `null` if it does not point to any node.

Your code will **only** be given the **head** of the original linked list.

Example 1:

Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

15.7K341125 Online

Accepted

All Submissions

Accepted 19 / 19 testcases passed

Sahil Khan submitted at Jul 17, 2026 18:31

₹583.25/mo Save over 58%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in on...

Runtime

0 ms Beats 100.00%

Memory

46.81 MB Beats 14.22%

Code | Java

```
1 /*
2 // Definition for a Node.
3 class Node {
4     int val;
5     Node next;
6     Node random;
7
8     public Node(int val) {
9         this.val = val;
10    }
```

View more

Code

JavaAuto

```
10     this.next = null;
11     this.random = null;
12 }
13 }
14 */
15
16 class Solution {
17     public Node copyRandomList(Node head) {
18         HashMap<Node, Node> map = new HashMap<>();
19
20         // Pass 1: create all copy nodes
21         Node curr = head;
22         while(curr != null){
23             map.put(curr, new Node(curr.val));
24             curr = curr.next;
25         }
26
27         // Pass 2: set next and random
28         curr = head;
29         while(curr != null){
30             map.get(curr).next = map.get(curr.next);
31             map.get(curr).random = map.get(curr.random);
32             curr = curr.next;
33         }
34
35         return map.get(head);
36     }
37 }
38 }
```

SavedLn 18, Col 9

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3